

PLANO DE TESTES ATUALIZADO – SERVEREST API

Responsável: Letícia Vidal | Versão: Final Challenge v7.0 (Completo)

1. 🎯 Objetivo do Projeto

O objetivo deste projeto é consolidar a estratégia de garantia de qualidade para a API ServeRest, realizando a transição de uma cobertura básica para uma suíte de testes automatizados de alta maturidade.

Através da utilização do Robot Framework em conjunto com técnicas de Engenharia de Prompt (GenAI), este desafio final visa:

- **Expandir a Cobertura de Testes:** Implementar mais de 40 cenários de teste, garantindo o cumprimento dos requisitos mínimos de 4 testes positivos, 4 negativos e 2 de contrato para cada um dos endpoints críticos (/usuarios, /login, /produtos, /carrinhos).
- **Validar a Integridade de Contratos e Regras:** Assegurar que as respostas da API estejam em conformidade com o esquema JSON esperado e que as regras de negócio (como controle de estoque e unicidade de usuários) sejam respeitadas.
- **Otimizar o Ciclo de Desenvolvimento (Shift-Left):** Utilizar Inteligência Artificial Generativa para acelerar a descoberta de cenários de borda (boundary values) e casos de erro complexos, reduzindo o tempo de design de testes.
- **Garantir Rastreabilidade e Manutenibilidade:** Estruturar o projeto seguindo padrões de Clean Code, com keywords reutilizáveis e massa de dados dinâmica (FakerLibrary), vinculando cada script automatizado a um ID de caso de teste no plano de planejamento.
- **Evidenciar a Qualidade:** Fornecer artefatos técnicos transparentes (Matriz de Cobertura Antes vs. Depois e Relatórios de Execução) que comprovem a resiliência do software para a entrega final da Sprint 4.

2. 🧩 ESCOPO

Endpoints cobertos: /usuarios, /login, /produtos, /carrinhos.

3. 📊 Matriz de Cobertura (Antes vs Depois)

Endpoint	Status	Positivo	Negativo	Contrato	Regras
Geral	🔴 ANTES	1	1	0	1
/usuarios	🟢 DEPOIS	4	5	2	2
/login	🟢 DEPOIS	3	4	3	2
/produtos	🟢 DEPOIS	3	5	2	2
/carrinhos	🟢 DEPOIS	2	8	5	3

4. 📌 MATRIZ DE CASOS DE TESTE (ATUALIZADA)

👤 /usuarios

ID	Cenário	Pré-condição	Input	Resultado	Tipo
CT-U-01	Criar usuário com sucesso	Nenhuma	Válidos	201 + _id	Positivo
CT-U-02	Email duplicado	Usuário criado	Mesmo email	400 + message	Negativo
CT-U-03	Nome ausente	Nenhuma	Sem nome	400	Negativo
CT-U-04	Email inválido	Nenhuma	email inválido	400	Negativo
CT-U-05	Senha ausente	Nenhuma	sem password	400	Negativo
CT-U-06	Payload vazio	Nenhuma	{}	400	Negativo
CT-U-07	Administrador inválido	Nenhuma	admin=string	400	Contrato
CT-U-08	Criar usuário admin	Nenhuma	admin=true	201	Positivo
CT-U-09	Nome com caracteres especiais	Nenhuma	válido	201	Positivo
CT-U-10	Persistência do usuário	Usuário criado	GET /usuarios	Dados consistentes	Regra
CT-U-11	Não retornar senha no GET	Usuário criado	GET	sem password	Segurança

🔑 /login

ID	Cenário	Pré-condição	Input	Resultado	Tipo
CT-L-01	Login com sucesso	Usuário criado	válidas	200 + token	Positivo
CT-L-02	Senha inválida	Usuário criado	senha errada	401	Negativo
CT-L-03	Email inexistente	Nenhuma	email fake	401	Negativo
CT-L-04	Payload vazio	Nenhuma	{}	400	Negativo
CT-L-05	Campos ausentes	Nenhuma	sem email	400	Negativo
CT-L-06	Token presente no sucesso	Login válido	—	auth string	Contrato
CT-L-07	Token ausente no erro	Login inválido	—	sem auth	Contrato
CT-L-08	Usar token endpoint protegido	Login feito	GET prot.	200	Integração
CT-L-09	Login admin vs comum	Usuários dif.	credenciais	200	Regra

📦 /produtos

ID	Cenário	Pré-condição	Input	Resultado	Tipo
CT-P-01	Criar produto como admin	Login admin	válidos	201 + _id	Positivo
CT-P-02	Usuário comum tentar criar	Login comum	válidos	403	Negativo
CT-P-03	Nome duplicado	Produto existe	mesmo nome	400	Negativo
CT-P-04	Nome vazio	Nenhuma	nome=""	400	Negativo
CT-P-05	Preço negativo	Nenhuma	preco=-10	400	Negativo
CT-P-06	Descrição muito longa	Nenhuma	texto grande	<500 chars	Robustez
CT-P-07	Validar contrato sucesso	Criação válida	—	_id presente	Contrato
CT-P-08	Ausência campos indevidos	Criação válida	—	campos esp.	Contrato
CT-P-09	Preço zero	Nenhuma	preco=0	201*	Regra
CT-P-10	Quantidade zero	Nenhuma	qtd=0	201*	Regra

ID	Cenário	Pré-condição	Input	Resultado	Tipo
CT-C-01	Criar carrinho com sucesso	Login + prod	válido	201	Positivo
CT-C-02	Criar carrinho duplicado	Já possui	válido	400	Negativo
CT-C-03	Sem token	Nenhuma	válido	401	Negativo
CT-C-04	Token inválido	Nenhuma	token fake	401	Negativo
CT-C-05	Produto inexistente	Login	id fake	400	Negativo
CT-C-06	Quantidade zero	Login	qtd=0	400	Negativo
CT-C-07	Quantidade negativa	Login	qtd=-1	400	Negativo
CT-C-08	Estoque insuficiente	Estoque baixo	qtd alta	400	Regra
CT-C-09	Produtos vazio	Login	[]	400	Contrato
CT-C-10	Campo produtos ausente	Login	{}	400	Contrato
CT-C-11	idProduto ausente	Login	sem id	400	Contrato
CT-C-12	quantidade ausente	Login	sem qtd	400	Contrato
CT-C-13	Tipo inválido quantidade	Login	"2" (str)	400	Contrato
CT-C-14	Concluir compra sem carrinho	Login	—	400	Regra

5. Evidência GenAI (Amazon Q)

Context: I am testing the POST /produtos endpoint of the ServeRest API. Current coverage: successful creation... Task: Generate additional negative scenarios... Focus on: boundary values, invalid types.

6. Check dos Requisitos e Boas Práticas

- ✓ +10 novos testes: ATENDIDO (44 totais) | ✓ 4 positivos, 4 negativos, 2 contrato por rota.
- ✓ Boas práticas: Keywords reutilizáveis, FakerLibrary (massa dinâmica), Asserts claros.
- ✓ Organização: Camadas (Resources, Data Factory, Base API).
- ✓ Git: Branch feature/challenge04-final-sprint4 | Commits claros.